캡스톤디자인2

Research Project

10주차

intRo-Lab.

청주대학교 무인항공기학과
김상현 교수 (www.intro-lab.com)

◆ 상태방정식

- 복잡한 동적 시스템(Dynamic System)의 입력과 출력, 내부 상태 간의 관계를 1차 미분 행렬 방정식으로 정식화한 현대 제어 이론의 핵심 도구

◆ 상태방정식의 등장

- 1950년대 이전에 사용된 고전 제어 이론은 라플라스 변환(Laplace Transform)을 이용한 전달함수 모델을 주로 사용.
- 전달함수를 이용한 자동제어 기법의 단점
  - 초기 조건 무시: 라플라스 변환을 적용하려면 모든 초기 조건이 0이라고 가정. 그러나 실제 드론과 같이 움직이는 시스템에서는 이동하는 도중에 자동제어 필요하게 됨. 즉, 초기 조건이 0이 아닌 상황에서의 제어가 필수적임.
  - SISO(단일입력 단일출력) 시스템에만 국한: 입력과 출력이 각각 하나인 단순한 시스템에는 유리하지만, 드론과 같이 수많은 모터와 센서가 있는 MIMO(다중입력 다중출력) 형태의 시스템에서는 전달함수로 표현하기가 극도로 복잡해짐.
- 이를 해결하기 위해 루돌프 칼만 등이 중심이 되어 시스템의 내부 물리량을 행렬로 묶어 다루는 상태방정식 중심의 현대 제어이론을 정립

◆ 상태방정식의 정의

- 선형 시불변 시스템의 연속시간 상태 공간 모델로 정의됨.
- 상태 공간(State Space): 시간에 따라 변하는 시스템 내부의 상태를 변수로 잡고, 이를 좌표축으로 설정하는 n차원의 수학적 공간을 의미. 예를 들어, 상태 변수를 위치와 속도로 정의하면 상태 공간의 x축은 위치 좌표가 되고, y축은 속도 좌표가 됨.
- 상태 변수(State Variable): 특정 시점에서 시스템의 물리적 상태를 완전하게 정의하는데 필요한 변수의 집합. 예를 들어, 달리는 자동차는 상태 변수로 위치와 속도를 사용할 수 있음.

$$\dot{\mathrm{x}}(t) = \mathrm{A}\mathrm{x}(t) + \mathrm{B}\mathrm{u}(t)$$
$$\mathrm{y}(t) = \mathrm{C}\mathrm{x}(t)$$

$\mathrm{x}(t)$: 상태 변수 $\mathrm{y}(t)$: 출력 변수

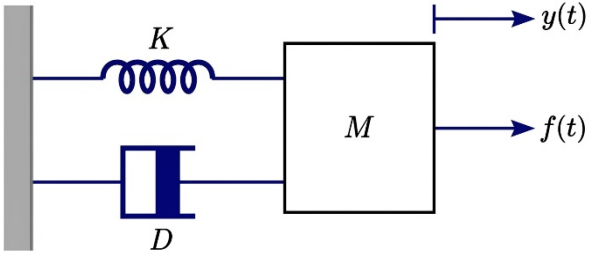
$\mathrm{u}(t)$: 입력 변수 C: 출력 행렬

A: 시스템 행렬

B: 입력 행렬

◆ 상태방정식 모델링 예시

● 동적 시스템 형상



● 미분방정식

$$M\frac{d^2}{dt^2}y(t) + D\frac{d}{dt}y(t) + Ky(t) = f(t)$$

◆ 상태방정식 모델링 예시

● 상태 변수 정의

$$y(t) = x_1(t) \quad \leftarrow \text{위치}$$

$$\dot{y}(t) = x_2(t) = \dot{x}_1(t) \quad \leftarrow \text{속도}$$



$$\mathrm{x}(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} \quad \leftarrow \text{상태 변수 벡터}$$



$$\dot{\mathrm{x}}(t) = \begin{bmatrix} \dot{x}_1(t) \\ \dot{x}_2(t) \end{bmatrix} \quad \leftarrow \text{상태 변수 벡터 미분}$$

◆ 상태방정식 모델링 예시

● 미분방정식을 상태 변수로 재정의

$$M\frac{d^2}{dt^2}y(t) + D\frac{d}{dt}y(t) + Ky(t) = f(t)$$

$$y(t) = x_1(t)$$

$$\frac{d}{dt}y(t) = \dot{y}(t) = \dot{x}_1(t) = x_2(t)$$

$$\frac{d^2}{dt^2}y(t) = \ddot{y}(t) = \dot{x}_2(t) = -\frac{K}{M}x_1(t) - \frac{D}{M}x_2(t) + \frac{1}{M}f(t)$$

$$M\dot{x}_2(t) + Dx_2(t) + Kx_1(t) = f(t)$$

◆ 상태방정식 모델링 예시

● 상태 변수 벡터를 이용한 상태방정식

$$\begin{bmatrix}\dot{x}_1(t)\\ \dot{x}_2(t)\end{bmatrix} = \begin{bmatrix}0 & 1\\ -\frac{K}{M} & -\frac{D}{M}\end{bmatrix}\begin{bmatrix}x_1(t)\\ x_2(t)\end{bmatrix} + \begin{bmatrix}0\\ \frac{1}{M}\end{bmatrix} f(t)$$

$$y(t) = [1 \quad 0]\begin{bmatrix}x_1(t)\\ x_2(t)\end{bmatrix}$$

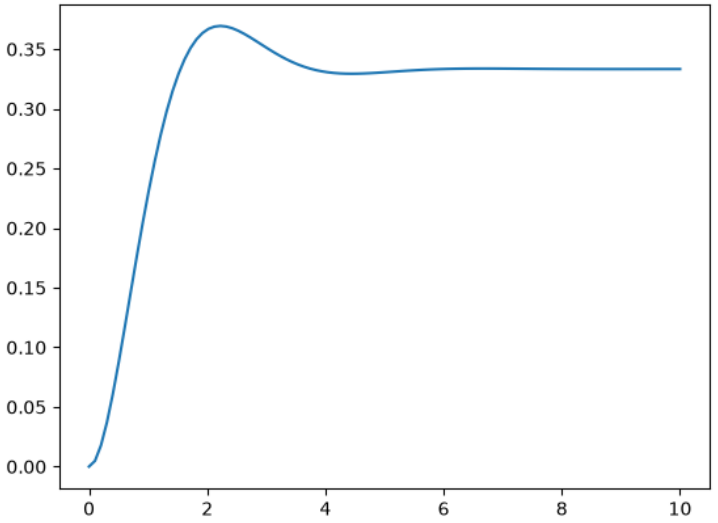


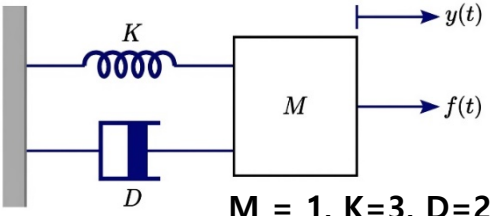
$$\dot{\mathrm{x}}(t) = \mathrm{A}\mathrm{x}(t) + \mathrm{B}f(t)$$

$$y(t) = \mathrm{C}\mathrm{x}(t)$$

◆ 상태방정식을 이용한 위치 변화 시뮬레이션

● 파이썬 코드





M = 1, K=3, D=2

```python
import control as ct
import matplotlib.pyplot as plt
import numpy as np

A = np.array([[ 0,  1],
              [-3, -2]])
B = np.array([[0],
              [1]])
C = np.array([1, 0])

sys = ct.ss(A, B, C, 0)

t, y = ct.step_response(sys, T=10)

plt.plot(t, y)
plt.show()
```

◆ 상태방정식을 이용한 위치 변화 시뮬레이션

●파이썬 코드

```
import control as ct
import matplotlib.pyplot as plt
import numpy as np

A = np.array([[ 0,  1],
              [-3, -2]])
B = np.array([[0],
              [1]])
C = np.array([1, 0])

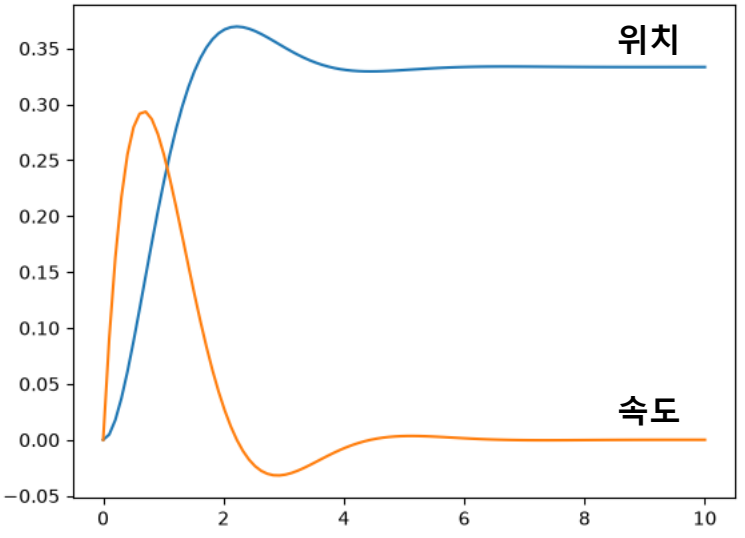
sys = ct.ss(A, B, C, 0)

t, y = ct.step_response(sys, T=10)

plt.plot(t, y)
plt.show()
```

◆ 상태방정식을 이용한 위치 변화 시뮬레이션

● 파이썬 코드: 위치, 속도 모두 출력 가능 → MIMO 구현 가능




```python
import control as ct
import matplotlib.pyplot as plt
import numpy as np

A = np.array([[ 0,  1],
              [-3, -2]])
B = np.array([[0],
              [1]])
C = np.array([[1, 0],
              [0, 1]])

sys = ct.ss(A, B, C, 0)

t, y = ct.step_response(sys, T=10)
print(y)
plt.plot(t, y[0,0,:], t, y[1,0,:])
plt.show()
```

◆ 상미분방정식(ODE, Ordinary Differential Equation)

- 독립변수가 1개인 미분방정식
- 예시

$$\frac{dy}{dx} = 2x$$

$$\frac{dy}{dx} = 3x + y$$

$$\frac{dy}{dx} + y = 3x + 1$$

1계 상미분방정식(1st Order ODE)

$$\frac{d^2y}{dx^2} + 2\frac{dy}{dx} + 3y = 0$$

$$\frac{d^2y}{dx^2} + \frac{dy}{dx} + y = x^2 + 4x + 1$$

2계 상미분방정식(1st Order ODE)

◆ 편미분방정식(PDE, Partial Differential Equation)

- 독립변수가 2개 이상인 미분방정식
- 예시

$$\frac{\partial z}{\partial x} = x$$

$$\frac{\partial z}{\partial x} + \frac{\partial z}{\partial y} = 0$$

$$\frac{\partial y}{\partial x} + y = 3x + 1$$

1계 편미분방정식(1st Order PDE)

$$\frac{\partial^2 z}{\partial y^2} + 2\frac{\partial z}{\partial x} + 3z = 0$$

$$\frac{\partial^2 z}{\partial x^2} + \frac{\partial z}{\partial y} + z = x^2 + 4y + 1$$

2계 편미분방정식(1st Order PDE)

◆ 오일러 방법(Euler's Method)

- 미분방정식을 컴퓨터로 풀기 위해 사용하는 가장 기초적인 수치 적분(Numerical Integration) 알고리즘
- 주어진 1계 상미분방정식과 초기 조건이 있을 때, 아주 작은 간격을 설정하고 다음 공식에 따라 점진적으로 해를 구함

$$\frac{dx}{dt} = f(t, x)$$

$$x(t_0) = x_0$$

$$t_{k+1} = t_k + \Delta t$$

$$x_{k+1} = x_k + \Delta t \cdot f(t_k, x_k)$$

◆ 오일러 방법(Euler's Method)

● 예시

$$\frac{dx}{dt} = f(t,x) = 1$$

$$t_0 = 0,\ \ x(t_0) = x_0 = 0,\ \ \ \Delta t = 0.1$$

$$t_1 = t_0 + \Delta t = 0 + 0.1 = 0.1$$
$$x_1 = x_0 + \Delta t \cdot 1 = 0 + 0.1 \cdot 1 = 0.1$$

$$t_2 = t_1 + \Delta t = 0.1 + 0.1 = 0.2$$
$$x_2 = x_1 + \Delta t \cdot 1 = 0.1 + 0.1 \cdot 1 = 0.2$$

$$t_{k+1} = t_k + \Delta t$$
$$x_{k+1} = x_k + \Delta t \cdot 1$$

◆ 오일러 방법(Euler's Method)

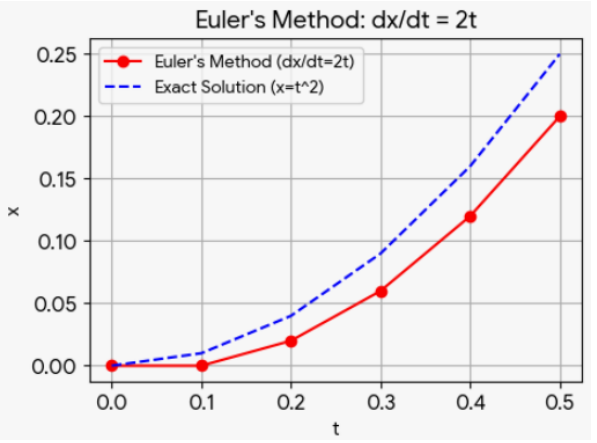
● 예시

$$\frac{dx}{dt} = f(t,x) = 2t$$

$$t_0 = 0, \quad x(t_0) = x_0 = 0, \quad \Delta t = 0.1$$

$$t_1 = t_0 + \Delta t = 0 + 0.1 = 0.1$$
$$x_1 = x_0 + \Delta t \cdot 2t_0 = 0 + 0.1 \cdot 2 \cdot 0 = 0$$
$$t_2 = t_1 + \Delta t = 0.1 + 0.1 = 0.2$$
$$x_2 = x_1 + \Delta t \cdot 2t_1 = 0 + 0.1 \cdot 2 \cdot 0.1 = 0.02$$

$$t_{k+1} = t_k + \Delta t$$
$$x_{k+1} = x_k + \Delta t \cdot 2t_k$$



◆ 수정 오일러 방법(Modified Euler's Method)

- 2차 룽게-쿠타 방법(RK2) 혹은 호인의 방법이라고도 불린다.
- 오일러 방법은 시작점의 기울기만 사용하는데, 수정 오일러 방법은 시작점과 끝점의 기울기를 각각 구하고 두 값의 평균 기울기 값을 이용하여 다음 단계 계산

$$\frac{dx}{dt} = f(t,x), \qquad x(t_0) = x_0$$

$$t_{k+1} = t_k + \Delta t$$

$$\tilde{x}_{k+1} = x_k + \Delta t \cdot f(t_k, x_k)$$

$$x_{k+1} = x_k + \Delta t \cdot \frac{f(t_k, x_k) + f(t_{k+1}, \tilde{x}_{k+1})}{2}$$

◆ 수정 오일러 방법(Modified Euler's Method)

● 예시

$$\frac{dx}{dt} = f(t,x) = 2t$$

$$t_0 = 0,\ \ x(t_0) = x_0 = 0,\ \ \ \Delta t = 0.1$$

$$t_1 = t_0 + \Delta t = 0 + 0.1 = 0.1$$
$$x_1 = x_0 + \Delta t \cdot (t_0 + t_1) = 0 + 0.1 \cdot (0 + 0.1) = 0.01$$
$$t_2 = t_1 + \Delta t = 0.1 + 0.1 = 0.2$$
$$x_2 = x_1 + \Delta t \cdot (t_1 + t_2) = 0.01 + 0.1 \cdot (0.1 + 0.2) = 0.04$$

$$t_{k+1} = t_k + \Delta t$$
$$x_{k+1} = x_k + \Delta t \cdot \frac{2t_k + 2t_{k+1}}{2} = x_k + \Delta t \cdot (t_k + t_{k+1})$$

RK4(Runge-Kutta 4th-Order Method, 4차 룽게-쿠타 방법)

- 가장 널리 쓰이는 수치해석 알고리즘
- 4개의 기울기를 구해서 이들의 가중 평균 기울기를 이용하여 다음 단계 계산

$$\frac{dx}{dt} = f(t,x), \qquad x(t_0) = x_0$$

$$t_{k+1} = t_k + \Delta t$$

$$k_1 = f(t_k, x_k)$$

$$k_2 = f\left(t_k + \frac{\Delta t}{2}, x_k + \frac{\Delta t}{2} \cdot k_1\right)$$

$$k_3 = f\left(t_k + \frac{\Delta t}{2}, x_k + \frac{\Delta t}{2} \cdot k_2\right)$$

$$k_4 = f(t_k + \Delta t, x_k + \Delta t \cdot k_3)$$

$$x_{k+1} = x_k + \Delta t \cdot \frac{k_1 + 2k_2 + 2k_3 + k_4}{6}$$

```python
import matplotlib.pyplot as plt
import numpy as np

# 1. 상태방정식 행렬 정의 (A, B, C, D)
# 2차 시스템 예제 (위치 및 속도)
A = np.array([[0, 1], [-3, -2]], dtype=float)
B = np.array([[0], [1]], dtype=float)
C = np.array([[1, 0], [0, 1]], dtype=float)
D = np.array([[0], [0]], dtype=float)

# 2. 오일러 방식 및 시뮬레이션 파라미터 설정
dt = 0.01  # 시간 간격 (Δt)
T_total = 10.0  # 총 시뮬레이션 시간
time = np.arange(0, T_total, dt)
n_steps = len(time)

# 3. 초기 상태 및 입력 설정
# x = [위치(x1), 속도(x2)]^T 초기값은 0
x = np.array([[0.0], [0.0]], dtype=float)
u = 1.0  # 단위 계단 입력 (Step Input)

# 결과를 저장할 배열
y_history = np.zeros((n_steps, 2))

# 4. 오일러 방식 수치 해석 루프
for i in range(n_steps):
  # 출력 방정식 계산: y = C*x + D*u
  y = C @ x + D * u
  y_history[i] = y.flatten()

  # 상태 방정식 미분 계산: x_dot = A*x + B*u
  x_dot = A @ x + B * u

  # 오일러 방식 상태 업데이트: x_{k+1} = x_k + Δt * x_dot
  x = x + dt * x_dot

# 5. 결과 시각화
plt.figure(figsize=(10, 5))
plt.plot(time, y_history[:, 0], label="Position, y1", linewidth=2)
plt.plot(time, y_history[:, 1], label="Velocity, y2", linewidth=2)
plt.title("State-Space Simulation using Euler's Method", fontsize=14)
plt.xlabel("Time (s)", fontsize=12)
plt.ylabel("Response", fontsize=12)
plt.axhline(0, color="gray", linestyle="--", linewidth=0.8)
plt.grid(True)
plt.legend()
plt.show()
```